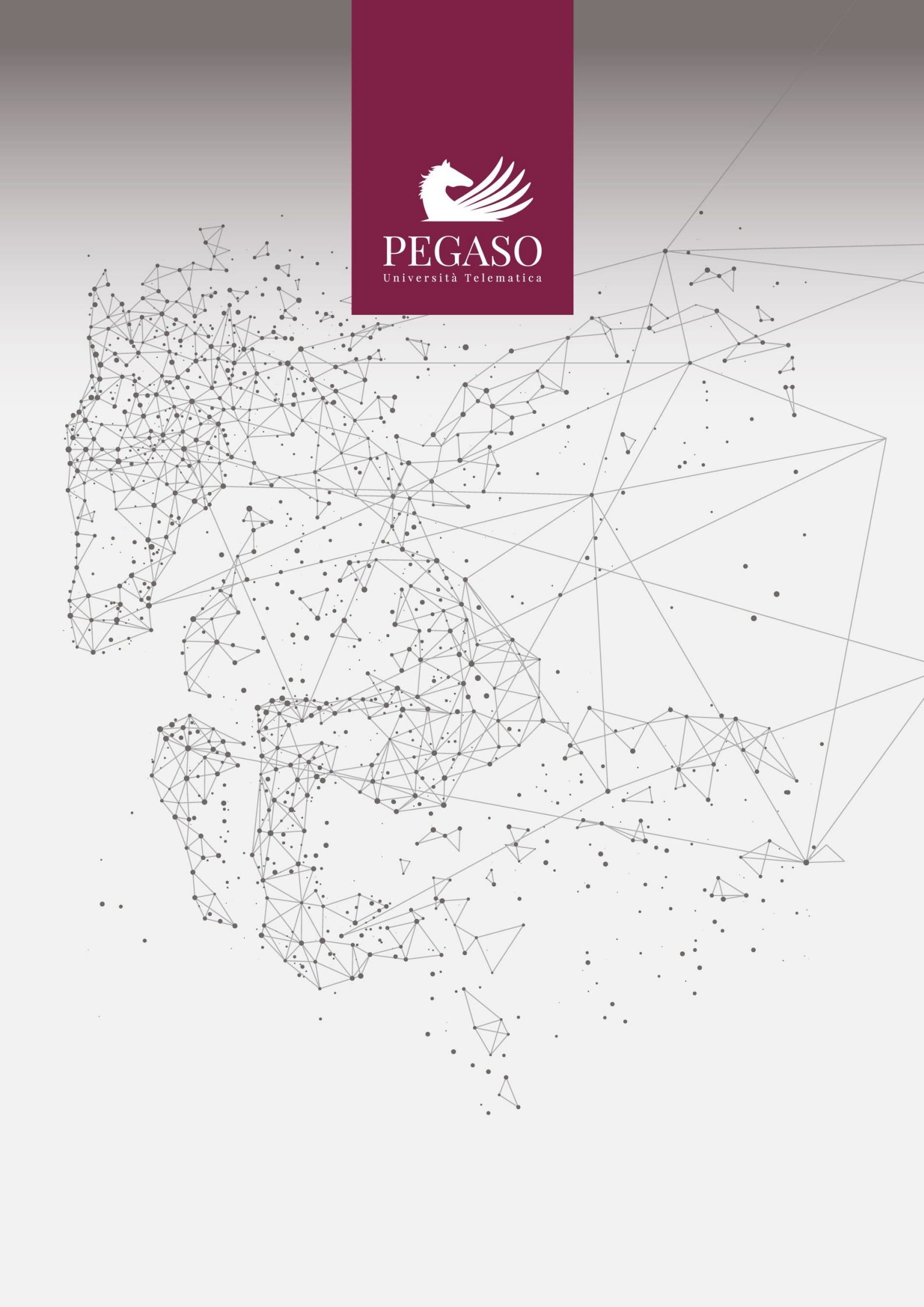

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. ESERCITAZIONI SU FUNZIONI 4

2.1 ESERCITAZIONE 1 – FUNZIONI CON PIÙ VALORI 4
2.2 ESERCITAZIONE 2 – FORMATTAZIONE LOCALE 5

3. ESERCITAZIONI SU FILE 7

3.1 ESERCITAZIONE 3 – DIZIONARIO DA FILE 7

4. ESERCITAZIONI SU STRINGHE 10

4.1 ESERCITAZIONE 4 – CONTROLLO PASSWORD 10

5. ESERCITAZIONI SU STRUTTURE DATI 12

5.1 ESERCITAZIONE 5 – RICERCA IN LISTA NUMEROSA 12
5.2 ESERCITAZIONE 6 – RICERCA IN LISTA E IN SET 13
5.3 ESERCITAZIONE 7 – ELIMINAZIONE DUPLICATI CON I SET 14
5.4 ESERCITAZIONE 8 – DIZIONARIO CON CHIAVI SEQUENZIALI 15
5.5 ESERCITAZIONE 9 – UNIONE DI DUE SEQUENZE PER UN DIZIONARIO 16
5.6 ESERCITAZIONE 10 – CARTE DA POKER 16

BIBLIOGRAFIA 19

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustrano ed analizzano alcune esercitazioni molto importanti che integrano i concetti e gli esercizi svolti finora. In particolare, si riportano due esercitazioni sulle funzioni (una per mostrare la restituzione di più valori di una funzione e una per mostrare come formattare secondo le convenzioni locali numeri e valute), una esercitazione sui dizionari creati a partire da file di testo, una esercitazione sulle stringhe per il controllo di un insieme di esempio di vincoli sulle password e sei esercitazioni sulle strutture dati. Le esercitazioni sulle strutture dati esaminano diversi aspetti come i tempi di ricerca, l'eliminazione di elementi duplicati, la creazione di dizionari con chiavi sequenziali, la creazione di dizionari come unione di due sequenze, la simulazione di un mazzo di carte da poker.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Esercitazioni su funzioni

Gli esempi di funzioni visti finora restituiscono un solo valore. Con Python ci sono due modi per restituire più valori:

- tramite una struttura dati (la funzione invocante può scorrere la collezione restituita);
- direttamente con più valori (a differenza degli altri linguaggi di programmazione come C++ o Java).

2.1 Esercitazione 1 – Funzioni con più valori

Scrivere una funzione che, passati come argomenti due numeri, restituisce il risultato di tutte le operazioni aritmetiche di base tra i due numeri:

1. addizione;
2. sottrazione;
3. moltiplicazione;
4. divisione.

Come è possibile osservare dal codice sorgente che segue, i valori possono essere restituiti separati da virgola. Parallelamente, nella funzione invocante, ci dovranno essere altrettante variabili/oggetti che raccolgono i risultati restituiti.

```
def op_aritmetiche(x,y):
    a = x+y
    b = x-y
    c = x*y
    if y!=0:
        d = x/y
    else:
        d=-1
    return a,b,c,d

def main():
    a,b,c,d = op_aritmetiche(7,5)
    print("Risultato dell'addizione tra 7 e 5:",a)
    print("Risultato della sottrazione tra 7 e 5:",b)
    print("Risultato della moltiplicazione tra 7 e 5:",c)
    print("Risultato della divisione tra 7 e 5:",d)

main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'elaborazione del codice precedente risulta essere:

```
Risultato dell'addizione tra 7 e 5: 12
Risultato della sottrazione tra 7 e 5: 2
Risultato della moltiplicazione tra 7 e 5: 35
Risultato della divisione tra 7 e 5: 1.4
```

2.2 Esercitazione 2 – Formattazione locale

In Python si usa la virgola come separatore per le migliaia e il punto come separatore dei decimali. Ciò deriva dal fatto che Python è stato sviluppato negli Stati Uniti. Per utilizzare le convenzioni italiane dei separatori numerici è necessario usare un modulo della stdlib: il **modulo locale**. Il modulo locale contiene le funzioni per l'internazionalizzazione dei programmi.

Procediamo, dunque, con la scrittura di un programma che prende in input un numero decimale scritto nel formato italiano.

La prima funzione da utilizzare è **setlocale**() che imposta i parametri di localizzazione. Si comunica, quindi, a Python il formato numerico della nazione desiderata secondo il seguente formato generale:

```
import locale
locale.setlocale(categoria, 'località')
```

La categoria usata più spesso è LC_ALL, cioè tutte le categorie di localizzazione. Altre categorie utilizzate meno spesso sono LC_NUMERIC per i soli numeri e LC_MONETARY per la sola valuta.

La località, invece, è una stringa costante tra apici singoli che identifica la nazione di localizzazione. Esempi di località sono:

- 'it_IT' per Italia;
- 'fr_FR' per Francia;
- 'de_DE' per Germania.

Nel caso di localizzazione italiana, la funzione setlocale() va chiamata nel seguente modo:

```
import locale
locale.setlocale(locale.LC_ALL, 'it_IT')
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Anche per il casting dei numeri inseriti come input è necessario utilizzare la funzione **atof**().

Tale funzione converte in float le stringhe in formato locale secondo la localizzazione determinata dalla funzione setlocale().

Ad ogni modo, come è possibile osservare attraverso l'elaborazione del seguente codice sorgente, l'output non è formattato correttamente:

```
import locale
locale.setlocale(locale.LC_ALL, 'it_IT')

a = locale.atof(input("Inserire numero: "))
print(type(a))
print(locale.format_string('%.2f',a))
```

Il risultato dell'elaborazione è, infatti, 3.5 invece di 3,5:

```
Inserire numero: 3,5
<class 'float'>
3.5
```

La funzione **format_string**() del modulo locale permette di formattare l'output in base alla localizzazione. Tale funzione è leggermente diversa dalla funzione format() vista finora (si usa, infatti, il simbolo di %):

```
import locale
locale.setlocale(locale.LC_ALL, 'it_IT')

a = locale.atof(input("Inserire numero: "))
print(type(a))
print(a)
```

Il risultato dell'elaborazione del codice precedente risulta essere:

```
Inserire numero: 3,5
<class 'float'>
3,50
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Esercitazioni su file

3.1 Esercitazione 3 – Dizionario da file

Si crei un dizionario di numeri telefonici a partire da un file di testo. Il file di testo di partenza può essere strutturato in due modi:

1. righe pari per i nominativi e righe dispari per i numeri come indicato di seguito:

```
Gioele Salvini
1234
Fabio Peruzzi
5678
Rembrandt Lombroso
9012
Pina Trobbiani
3456
Etta Sgarbi-Zetticci
7890
Ida Cignaroli
1234
```

2. nominativi e numeri sulla stessa riga separati da un carattere (es. virgola, punto e virgola) – stile file csv (comma-separated values) utilizzato per importazione ed esportazione di tabelle di dati come indicato di seguito:

```
Gioele Salvini,1234
Fabio Peruzzi,5678
Rembrandt Lombroso,9012
Pina Trobbiani,3456
Etta Sgarbi-Zetticci,7890
Ida Cignaroli,1234
```

Andiamo a scrivere, quindi, due diversi codici sorgente che risolvono i due formati.

Per quanto riguarda il primo formato, dopo l'inizializzazione di un dizionario vuoto e l'apertura del file di testo in lettura, si procede subito alla lettura della prima riga. Se tale riga non è vuota, si procede a "ripulire" la stringa e a copiarla in chiave. Successivamente, viene letta la riga successiva (corrispondente al numero telefonico) e viene copiata in valore. Si procede, dunque, con l'aggiunta della coppia chiave-valore nel dizionario d. Infine, prima di chiudere il corpo del ciclo si legge la riga successiva corrispondente, eventualmente, ad un'altra persona (se riga non vuota). Si noti che le letture di riga all'interno del corpo del

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ciclo sono sempre pari perché 2 sono le informazioni relative alla stessa persona memorizzate in righe diverse.

Infine, per prova, si procede con la visualizzazione del numero di una persona presente nel dizionario.

```
def main():
    d = dict()
    infile = open("rubrical.txt",'r')
    riga = infile.readline()

    while riga!='':
        chiave = riga.rstrip()
        valore = infile.readline().rstrip()
        d[chiave] = valore
        riga = infile.readline()

    infile.close()

    print(d)
    print("Il numero di Fabio Peruzzi è",d["Fabio Peruzzi"])

main()
```

Il risultato dell'elaborazione precedente risulta essere:

```
{'Gioele Salvini': '1234', 'Fabio Peruzzi': '5678', 'Rembrandt Lombroso': '9012'
, 'Pina Trobbiani': '3456', 'Etta Sgarbi-Zetticci': '7890', 'Ida Cignaroli': '12
34'}
Il numero di Fabio Peruzzi è 5678
```

Si procede, ora, con la scrittura del codice sorgente che risolve lo stesso problema, ma per un diverso formato del file di testo in memoria. Si procede con l'inizializzazione di un dizionario vuoto e l'apertura del file di testo in lettura. Per ciascuna riga del file, viene richiamato il metodo split (con la virgola come parametro passato alla funzione) che restituisce una lista composta da due valori, rispettivamente chiave e valore del dizionario. Si procede, quindi, al prelievo dei due elementi e all'aggiunta della coppia chiave-valore al dizionario.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():
    d = dict()
    infile = open("rubrica2.txt",'r')

    for riga in infile:
        r = riga.rstrip().split(',')
        chiave = r[0]
        valore = r[1]
        d[chiave]=valore

    infile.close()

    print(d)
    print("Il numero di Fabio Peruzzi è",d["Fabio Peruzzi"])

main()
```

Il risultato dell'elaborazione precedente risulta essere identico al caso precedente:

```
{'Gioele Salvini': '1234', 'Fabio Peruzzi': '5678', 'Rembrandt Lombroso': '9012'
, 'Pina Trobbiani': '3456', 'Etta Sgarbi-Zetticci': '7890', 'Ida Cignaroli': '12
34'}
Il numero di Fabio Peruzzi è 5678
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Esercitazioni su stringhe

4.1 Esercitazione 4 – Controllo password

Si vuole realizzare un programma[1] che controlla le password di un sistema informatico. I requisiti che la password deve soddisfare sono:

1. password di lunghezza almeno di 7 caratteri;
2. deve contenere almeno una lettera maiuscola;
3. deve contenere almeno una lettera minuscola;
4. deve contenere almeno un numero.

Progettiamo, dunque, una funzione valida_password() che restituisce True o False per indicare se la password è valida o no. Nella funzione verranno utilizzati diversi flag:

- ha_maiuscole per verificare la presenza di una lettera maiuscola;
- ha_minuscole per verificare la presenza di una lettera minuscola;
- ha_numeri per verificare la presenza di una cifra numerica.

Per il solo vincolo della lunghezza, non vengono usati flag perché la lunghezza inferiore ai 7 caratteri è condizione necessaria e sufficiente a ritenere la password non valida.

Al termine della funzione, con una struttura decisionale e una serie di operatori di and logico, sarà possibile verificare il soddisfacimento di tutte e tre vincoli.

[1] Esercizio tratto dal libro Introduzione a Python, T. Gaddis, PEARSON – pag. 302.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def valida_password(p):
    ha_maiuscole = False
    ha_minuscole = False
    ha_numeri = False

    if len(p)<7:
        return False

    for carattere in p:
        if carattere.isupper():
            ha_maiuscole = True
        if carattere.islower():
            ha_minuscole = True
        if carattere.isdigit():
            ha_numeri = True

    if ha_maiuscole and ha_minuscole and ha_numeri:
        return True
    else:
        return False
```

La funzione main che permette di raccogliere la password e richiamare la funzione valida_password() è la seguente:

```
def main():
    password = input("Inserire password ")
    if valida_password(password):
        print("La password è valida")
    else:
        print("La password non è valida")
```

Il risultato dell'elaborazione per la password di esempio "hhhh" risulta essere il seguente:

```
Inserire password hhhh
La password non è valida
```

Il risultato dell'elaborazione per la password di esempio "Ab1ccccc" risulta essere il seguente:

```
Inserire password Ab1ccccc
La password è valida
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Esercitazioni su strutture dati

Le strutture dati più utilizzate in Python sono:

- liste;
- tuple;
- set;
- dizionari.

La scelta della struttura dati per la risoluzione di un determinato problema è di fondamentale importanza. Gli aspetti che devono essere presi in considerazione per la scelta di una struttura dati sono diversi, ma uno dei più importanti è la **complessità di ricerca**. La complessità di ricerca, soprattutto per grandi quantità di dati, riveste un ruolo fondamentale per la scelta di una particolare struttura dati. Nelle liste e nelle tuple la ricerca è **lineare**:

- caso migliore: primo elemento della lista/tuple;
- caso peggiore: elemento non esistente nella lista dopo l'analisi di tutti gli elementi della lista.

L'uso di liste è una soluzione impraticabile per la memorizzazione di grandi quantità di dati soggetti a ricerche (si veda l'esercitazione n. 5).

Nei set, invece, la ricerca è **sublineare**. I set non sono sequenze (gli elementi del set non hanno un indice) e possono memorizzare al massimo una copia di ogni elemento. I set sono eccellenti per la ricerca (si veda l'esercitazione n. 6) e per l'eliminazione dei duplicati (si veda esercitazione n. 7).

Anche nei dizionari, che creano una mappa fra chiavi e valori, la ricerca è **sublineare**. I dizionari, in cui tutte le chiavi sono diverse (potenzialmente anche per tipo di dato), sono eccellenti per le ricerche chiave-valore.

Per i dizionari, inoltre, è possibile costruire un dizionario con valori sequenziali della chiave (si veda l'esercitazione n. 8) o come unione di due sequenze chiavi e valori (si veda l'esercitazione n. 9).

L'ultima esercitazione di questo capitolo, infine, riporta la modellazione di un mazzo di carte da poker e una possibile applicazione.

5.1 Esercitazione 5 – Ricerca in lista numerosa

Si vuole realizzare un programma di ricerca in una lista con molti elementi. Per poter mostrare tale proprietà, si utilizzerà il **modulo time** che permette di calcolare i tempi di elaborazione. In particolare, la funzione time() del

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

modulo time permette di salvare il timestamp in un determinato momento. Richiamando tale funzione prima e dopo l'elaborazione e calcolando la differenza tra i due timestamp, è possibile ottenere il tempo di elaborazione delle istruzioni contenute tra le due chiamate a time().

Si dichiara una lista di 100 milioni di elementi sequenziali (e, quindi, non ripetuti) ottenuti attraverso la funzione range. Vengono, dunque, ricercati due elementi: uno presente nella lista (0) e uno non presente (200.000.000).

```
import time
lista = list(range(100000000))
ric1=0
ric2= 200000000
start_time = time.time()
if ric1 in lista:
    print(ric1,"trovato nella lista")
else:
    print(ric1,"non trovato nella lista")
print("Ricerca di",ric1,"eseguita in", \
      format(time.time() - start_time,'.5f'),"secondi")
start_time = time.time()
if ric2 in lista:
    print(ric2,"trovato nella lista")
else:
    print(ric2,"non trovato nella lista")
print("Ricerca di",ric2,"eseguita in", \
      format(time.time() - start_time,'.5f'),"secondi")
```

Il risultato dell'elaborazione del programma precedente mostra il grande numero di secondi sia nel caso di presenza sia nel caso di assenza di un elemento in una lista.

```
0 trovato nella lista
Ricerca di 0 eseguita in 87.33321 secondi
200000000 non trovato nella lista
Ricerca di 200000000 eseguita in 266.68265 secondi
```

5.2 Esercitazione 6 – Ricerca in lista e in set

Si vuole realizzare un programma che mostri la differenza dei tempi di ricerca in una lista e quelli in un set. Come nel caso precedente, si inizializza una lista e un set grazie alla funzione range (in questo caso si utilizzeranno 10 milioni di elementi sequenziali e non ripetuti). Si procede alla ricerca in entrambe le strutture dati di un numero non presente nelle strutture dati stesse.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import time
lista = list(range(10000000))
insieme = set(lista)
ric= 200000000
start_time = time.time()
if ric in lista:
    print(ric,"trovato nella lista")
else:
    print(ric,"non trovato nella lista")
print("Ricerca di",ric,"eseguita in", \
      format(time.time() - start_time,'.5f'),"secondi")
start_time = time.time()
if ric in insieme:
    print(ric,"trovato nel set")
else:
    print(ric,"non trovato nel set")
print("Ricerca di",ric,"eseguita in", \
      format(time.time() - start_time,'.5f'),"secondi")
```

La differenza dei tempi di elaborazione è visibile nel risultato visualizzato a schermo ed è di un ordine di grandezza.

```
200000000 non trovato nella lista
Ricerca di 200000000 eseguita in 0.21355 secondi
200000000 non trovato nel set
Ricerca di 200000000 eseguita in 0.01067 secondi
```

5.3 Esercitazione 7 – Eliminazione duplicati con i set

Si vuole realizzare un programma che permetta l'eliminazione dei duplicati attraverso i set.

Si procede con la dichiarazione di una lista con elementi ripetuti. La lista viene visualizzata a schermo insieme al numero di elementi che contiene. Successivamente, si richiamano in modo combinato le funzioni set() e list() che permetteranno di ottenere una nuova lista che conterrà gli elementi della lista di partenza non ripetuti.

```
lista = ["casa","mare","mare","mare","montagna","casa","montagna"]
print("Lista iniziale:",lista)
print("Gli elementi della lista sono",len(lista))
newlista = list(set(lista))
print("Gli elementi della lista dopo la trasformazione in set sono",len(newlista))
print("Lista elaborata:",newlista)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione precedente, mostra che il numero degli elementi presenti nella lista passa da 7 a 3 perché 3 sono gli elementi non ripetuti.

```
Lista iniziale: ['casa', 'mare', 'mare', 'mare', 'montagna', 'casa', 'montagna']
Gli elementi della lista sono 7
Gli elementi della lista dopo la trasformazione in set sono 3
Lista elaborata: ['casa', 'mare', 'montagna']
```

5.4 Esercitazione 8 – Dizionario con chiavi sequenziali

Si vuole realizzare un programma che permette la costruzione di un dizionario con valori sequenziali della chiave. Dopo la dichiarazione di una lista di elementi (non importa se ripetuti o meno poiché tali valori corrisponderanno ai valori del dizionario), si richiama la funzione **enumerate**() passandogli la lista appena creata. L'oggetto risultante contiene una sequenza di tuple formate da due elementi:

1. il primo è il numero sequenziale partendo da 0;
2. il secondo è l'elemento della lista.

Successivamente, l'oggetto enumerate può essere passato alla funzione dict() per realizzare un dizionario con chiavi sequenziali.

```
lista = ["casa","mare","mare","mare","montagna","casa","montagna"]
e = enumerate(lista)
print("Risultato di enumerate(lista)",enumerate(lista))
for elemento in e:
    print(i)
dizionario = dict(enumerate(lista))
print("Il dizionario sarà:",dizionario)
```

Il risultato dell'elaborazione del programma precedente risulta essere:

```
Risultato di enumerate(lista) <enumerate object at 0x000002597C3EBEC0>
(0, 'casa')
(1, 'mare')
(2, 'mare')
(3, 'mare')
(4, 'montagna')
(5, 'casa')
(6, 'montagna')
Il dizionario sarà: {0: 'casa', 1: 'mare', 2: 'mare', 3: 'mare', 4: 'montagna',
5: 'casa', 6: 'montagna'}
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5.5 Esercitazione 9 – Unione di due sequenze per un dizionario

Si vuole realizzare un programma che permette la costruzione di un dizionario come unione di due sequenze: una di chiavi e una di valori. Per poter realizzare tale dizionario la sequenza delle chiavi e quella dei valori devono avere la stessa lunghezza. Ai fini dell'esercizio, si ricorda che anche le stringhe sono sequenze.

Nel codice sorgente che segue, la stringa chiavi è una sequenza di 7 caratteri (ciascuno dei quali corrisponderà ad una chiave), mentre la lista valori è una sequenza di stringhe. L'oggetto restituito dalla funzione **zip**() a cui vengono passate le liste *chiavi* e *lista_valori*, viene, poi, utilizzato come parametro alla funzione dict() che permetterà di ottenere il dizionario.

```
chiavi = "abcdefg"
lista_valori = ["casa","mare","mare","mare","montagna","casa","montagna"]
dizionario = dict(zip(chiavi,lista_valori))
print("Il dizionario sarà:",dizionario)
```

Il risultato dell'elaborazione precedente risulta essere:

```
Il dizionario sarà: {'a': 'casa', 'b': 'mare', 'c': 'mare', 'd': 'mare', 'e': 'm
ontagna', 'f': 'casa', 'g': 'montagna'}
```

5.6 Esercitazione 10 – Carte da poker

Si vuole realizzare un programma[2] che permette di utilizzare un dizionario per simulare un mazzo di carte. Nei giochi come il Blackjack, le carte da poker hanno i seguenti valori numerici:

- le carte numeriche hanno il loro valore nominale (es. il valore del 2 di picche è 2, il valore del 5 di quadri è 5);
- le figure hanno tutte un valore pari a 10;
- gli assi possono avere valore 1 o 11 a scelta del giocatore.

Il programma utilizzerà un dizionario per simulare un mazzo di carte da poker standard (a tutti gli assi verrà assegnato valore 1). Le coppie chiave-valore usano come chiave il nome della carta e come valore il relativo valore numerico (es. 'Regina di quadri':10).

Il programma chiede all'utente il numero di carte da estrarre ed estrae casualmente dal mazzo una mano costituita dal numero di carte specificato. Vengono visualizzati i nomi delle carte e il valore totale della mano.

Il programma sarà suddiviso in 3 funzioni:

[2] Esercizio tratto dal libro Introduzione a Python, T. Gaddis, PEARSON – pag. 322.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. main();
2. creare_carte();
3. distrib_carte().

Nella funzione main(), si richiama la funzione creare_carte() e si chiede all'utente quante carte si vogliono estrarre. Una volta noto il valore di num_carte si procede a richiamare la funzione distrib_carte() che stamperà a schermo le carte estratte e restituirà il valore totale della mano, che verrà, successivamente, visualizzato a schermo.

```
def main():
    m = creare_carte()
    num_carte = int(input("Quante carte devo estrarre? "))
    mano = distrib_carte(m, num_carte)
    print("Valore totale della mano",mano)
```

La funzione creare_carte() permette di definire il dizionario delle 52 carte da poker in cui la chiave corrisponde al nome della carta e il valore al valore della carta.

```
def creare_carte():
    m = {"Asso di picche":1,"2 di picche":2,"3 di picche":3,
         "4 di picche":4,"5 di picche":5,"6 di picche":6,
         "7 di picche":7,"8 di picche":8,"9 di picche":9,
         "10 di picche":10,"Fante di picche":10,
         "Regina di picche":10,"Re di picche":10,

         "Asso di cuori":1,"2 di cuori":2,"3 di cuori":3,
         "4 di cuori":4,"5 di cuori":5,"6 di cuori":6,
         "7 di cuori":7,"8 di cuori":8,"9 di cuori":9,
         "10 di cuori":10,"Fante di cuori":10,
         "Regina di cuori":10,"Re di cuori":10,

         "Asso di fiori":1,"2 di fiori":2,"3 di fiori":3,
         "4 di fiori":4,"5 di fiori":5,"6 di fiori":6,
         "7 di fiori":7,"8 di fiori":8,"9 di fiori":9,
         "10 di fiori":10,"Fante di fiori":10,
         "Regina di fiori":10,"Re di fiori":10,

         "Asso di quadri":1,"2 di quadri":2,"3 di quadri":3,
         "4 di quadri":4,"5 di quadri":5,"6 di quadri":6,
         "7 di quadri":7,"8 di quadri":8,"9 di quadri":9,
         "10 di quadri":10,"Fante di quadri":10,
         "Regina di quadri":10,"Re di quadri":10}

    return m
```

La funzione distrib_carte() riceve come parametri il dizionario delle carte e il numero di carte da estrarre. La funzione inizializza a zero un accumulatore della mano e verifica che il numero di carte da

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

estrarre sia inferiore o uguale al numero di carte presenti nel mazzo. Nel caso in cui fosse superiore, tale valore viene saturato a len(m) che è pari a 52.

Vengono, dunque, estratte *num* carte attraverso la chiamata al metodo popitem() dei dizionari che estrae casualmente ed elimina una coppia chiave-valore. L'eliminazione dal dizionario della coppia chiave-valore corrispondente ad una carta è importante per evitare l'estrazione ripetuta della stessa carta.

```
def distrib_carte(m,num):
    mano = 0
    if num>len(m):
        num=len(m)
    for c in range(num):
        carta,valore = m.popitem()
        print(carta)
        mano+=valore
    return mano
```

Il risultato dell'elaborazione della funzione main() che richiama le altre funzioni per un numero di carte da estrarre pari a 10 risulta essere il seguente:

```
Quante carte devo estrarre? 10
Re di quadri
Regina di quadri
Fante di quadri
10 di quadri
9 di quadri
8 di quadri
7 di quadri
6 di quadri
5 di quadri
4 di quadri
Valore totale della mano 79
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).